PRODUCT NOTE · RAZORPAY × ISB AI PM BUILD CHALLENGE · TRACK 2: RECOVER AND GROW WITH AI

Margin-Aware Cart Recovery

An Agent Studio agent that prices each abandoned cart's recovery offer for **profit kept, not orders won back**. Often the right offer is ₹0, and a control group proves the gain. By Anuraag Praharaj, PGP Co'27.

Live prototype → | Detailed deck (5 slides) → | Three synthetic D2C merchants · all numbers simulated unless labelled

1 Problem

Merchant: a D2C brand on Razorpay Magic Checkout selling several categories with very different margins (e.g. serums ~70%, protein ~45%).

- **Today:** when a shopper abandons checkout, a recovery offer is picked from the merchant's preset coupons (flat, or tiered by cart value) under a cap, and success is counted as **orders recovered**.
- **The leak:** the discount also goes to shoppers who would have returned on a reminder alone. Illustrative 1,000 abandoned carts, AOV ₹1,000, 40% margin: a reminder recovers 150 orders (₹60k). 10% off recovers 200 (+33%), but profit is still ₹60k.
- **The gap:** Razorpay's public material on Magic Checkout offers and the Agent Studio cart agent (which picks among preset offers up to the merchant's maximum) describes no control group, margin floor or incremental-profit report.

2 Solution: signal → decision → action

- **Signal:** Magic Checkout's abandoned-cart event plus the payment-attempt result. The flow runs only if the cart holds one of the merchant's **top-5 margin categories**.
- **10 bucket-level signals:** first-time/repeat, payment method, payment status (bank decline, UPI timeout, OTP failed), time of day, day type, season, region tier, cart size vs the store's AOV, coupon behaviour, traffic source (UTM), plus the categories in the cart.
- **Decision:** for every discount d from 0 to the cap (0.5% steps), predict the chance of purchase $c(d \mid x)$ and pick the d that maximises $c(d)\cdot(m - d)$, within the cap and margin floor.
- **Action:** a WhatsApp or email reminder with a **rupee saving rounded to ₹5**, applied only to eligible items. Failed payments get a one-tap retry link first.

Objective. Today's tools maximise orders, $\alpha + (1-\alpha)\cdot c(d)$, which rises with every extra % of discount. This agent maximises profit, $\alpha\cdot m + (1-\alpha)\cdot c(d)\cdot(m - d)$. α (straight-through conversion) doesn't depend on d, so only the abandoned branch is optimised.

3 Why AI, not a fixed rule

- **Pays only where it changes the outcome.** The model estimates each cart's no-discount buy chance and skips carts that would convert anyway, e.g. 77–91% of failed-payment carts get ₹0 + a retry link.
- **Too many combinations for a rule:** 10 correlated signals × category mix ≈ 47,600 cells. One model learns them jointly, holding the others fixed, and prices combinations it has rarely seen.
- **Finds the depth, not just yes/no:** response flattens as discounts grow, and each cart's peak differs.
- **Learns and proves:** random test offers keep the discount effect causal. A 10% reminder-only control measures the lift. Explore → learn → optimise (a contextual bandit in production).
- **Where GenAI helps:** maps product titles to categories, reads margin files (Excel/PDF), explains each recommendation in plain words, answers merchant questions.

4 Key decisions and what I left out

DECISION	WHY
Top-5 margin categories only	Low-margin items have no room to discount, and it keeps setup to one form
Merchant owns cap and floor	Pricing and the P&L are theirs. It bounds the worst case and matches Agent Studio's never-exceed-the-maximum rule
Randomised offers + 10% control	The only clean way to separate "the discount caused it" from "they'd have bought anyway"
Pooled by product type, margins private	New merchants start from Razorpay's existing data. Only bucket-level response is shared
First-time vs repeat kept	Repeat buyers convert ~2× on a reminder alone, so discounts mostly leak there. Customer-level, switchable
Rupees, not odd percentages	"Save ₹75" reads as real; "6.5% off" reads as arbitrary
Left out: device, income	Customer-level personal data (DPDP) and a proxy for spending power, so unfair
Left out: all categories, shopper histories, list-price changes, voice	No margin room · privacy · MRP law and fairness · time

5 Merchant control, trust and failure

- **Review · change · approve · stop:** review before going live; per-category caps and a margin floor; a switch per signal that shows its profit cost (region adds only ₹0.4–1k per 1,000 carts); any group set to reminder only; one-click pause; an audit log per cart with drivers and checks.
- **Next best actions** surface the decision with the profit at stake, e.g. "set failed payments to retry only".
- **Failure modes:** thin data → fall back to the merchant's rule · drift → stop-loss if a group falls below control · discount gaming → per-contact offer cap and random ₹0 offers · outage → reminder only, never a bigger discount. DPDP consent before any message.

6 Success metrics

North star: incremental profit per 1,000 abandoned carts = treated profit − control profit, after discounts. **Secondary:** discount ₹ per incremental order, recovery rate vs control. **Guardrails:** opt-out and complaint rate, margin-floor breaches = 0, returns on recovered orders.

SIMULATED, 30 DAYS, PER 1,000 CARTS	FLAT 10%	TIERED	AGENT
Glow & Co. · skincare	+₹37.5k	+₹41.7k	**+₹52.8k**
ThreadLine · apparel	+₹68.2k	+₹70.2k	**+₹88.0k**
FuelUp · sports nutrition	−₹15.1k	−₹10.6k	**+₹45.0k**

The agent recovers slightly fewer orders than flat 10% but spends 49–76% less on discounts. The results reflect how the simulation was built; real gains need the pilot below.

7 Assumptions → validation with real data

- Discount response differs by signal → 4-week pilot, 3 merchants, random offers, lift compared by group.
- Merchants can't measure incrementality today and will share margins → 10 merchant interviews; setup completion rate.
- Pooled curves transfer across stores → pooled vs store-only predictions on held-out carts.

Disclosures. No external APIs; all merchant data is synthetic. Built with Claude (research, critique, code, copy). Prototype uses open-source Chart.js, SheetJS and pdf.js (via jsDelivr) and Razorpay's Blade fonts. Sources: Razorpay docs and blog (Magic Checkout offers and abandoned-cart webhook; Agent Studio launch and guardrails), Baymard Institute (cart abandonment), MediaNama (Agent Studio coverage). Concept only, not a Razorpay product.